Инструмент визуализации графа зависимостей (Вариант №16)

1. Общее описание

Приложение выполняет автоматический анализ и построение графа зависимостей пакетов с использованием нерекурсивного алгоритма поиска в глубину (DFS). Поддерживаются два режима работы:

- **Режим реального репозитория**: сбор информации о прямых и транзитивных зависимостях пакета npm через HTTP API реестра `https://registry.npmjs.org`.
- **Режим тестирования (`test_mode`)**: загрузка графа из локального JSON-файла, где узлы обозначаются заглавными латинскими буквами (`A`, `B`, `C` и т.д.).

Граф генерируется в двух форматах:

1. Текстовое описание на языке диаграмм **D2** (`.d2`).
2. Векторное изображение в формате **SVG** (`.svg`), сформированное средствами стандартной библиотеки Python.

---

2. Описание всех функций и настроек

Конфигурационные параметры (`config.toml`)

Параметр	Тип	Описание
`package_name`	`string`	Имя анализируемого пакета (например, `"express"` или `"A"`)
`repo_url`	`string`	URL реестра npm или путь к локальному JSON-файлу тестового графа
`test_mode`	`boolean`	`true` для работы с тестовым графом, `false` для npm
`output_file`	`string`	Имя выходного файла визуализации (например, `"graph.svg"`)
`max_depth`	`integer`	Максимальная глубина анализа зависимостей (`>= 0`)
`filter_substring`	`string`	Подстрока для фильтрации/исключения пакетов из графа

Основные модули и функции проекта

- **`src/config.py`**:

- `load_config(file_path)`: Загружает и валидирует TOML-конфигурацию.
- `validate_config(config)`: Проверяет корректность типов и диапазона значений.
- `print_config(config)`: Выводит параметры в формате «ключ-значение».

- **`src/fetcher.py`**:
  - `fetch_direct_dependencies(package_name, repo_url, test_mode)`: Извлекает прямые зависимости пакета из npm registry API или JSON-файла.

- **`src/graph.py`**:
  - `build_graph_dfs(...)`: Строит граф зависимостей с помощью **нерекурсивного DFS** со стеком, учитывает `max_depth`, фильтрацию и обнаруживает циклы.
  - `compute_load_order(graph, start_pkg)`: Рассчитывает порядок загрузки пакетов методом обратного обхода (post-order DFS).

- **`src/visualizer.py`**:
  - `generate_d2_code(graph)`: Формирует текстовую диаграмму D2.
  - `generate_svg_image(graph, output_file, root_pkg)`: Генерирует валидный векторный SVG-файл с раскладкой по рангам.

---

3. Описание команд для сборки проекта и запуска тестов

Запуск всех тестов

```
python3 -m unittest discover -s tests
```

или через `run.sh` / `Makefile`:

```
./run.sh
```

или

```
make test
```

Очистка сгенерированных файлов

```
make clean
```

4. Примеры использования

Пример 1: Запуск в тестовом режиме (`test_config.toml`)

```
python3 -m src.main --config test_config.toml
```

Вывод консоли:

```
=== Configuration Parameters ===
package_name: A
repo_url: test_repo.json
test_mode: True
output_file: test_graph.svg
max_depth: 4
filter_substring: X
================================

Direct dependencies for 'A':
  - B
  - C

Built graph with 6 nodes:
  A -> [B, C]
  C -> [F]
  F -> [None]
  B -> [D, E]
  E -> [None]
  D -> [A]

Detected Cyclic Dependencies:
  [CYCLE WARNING] A -> B -> D -> A

Installation / Load Order for 'A':
  1. D
  2. E
  3. B
  4. F
  5. C
  6. A

Saved D2 markup to: test_graph.d2
Saved SVG visualization to: test_graph.svg
```

Пример 2: Запуск с выбором конкретного этапа

```
python3 -m src.main --config config.toml --stage 1
```

Сравнение с реальным менеджером пакетов (npm)

1. **Алгоритм npm**: npm при установке производит подъём (hoisting) и сплющивание (flattening) дерева в структуру `node_modules`, а также разрешает диапазон версий (semver).
2. **Наш инструмент**: осуществляет прямой нерекурсивный поиск в глубину (DFS) и строит строгое топологическое дерево/граф зависимостей.
